

Cryptography and Secure Development

1. Course Summary
2. Mock Exam

Course Summary

Course Outline

► Two main topics:

I. Cryptography

- Algorithms to encrypt and decrypt information
- Other security purposes: Random Numbers, Secret Sharing, Message Digest, Digital Signatures

II. Secure Development

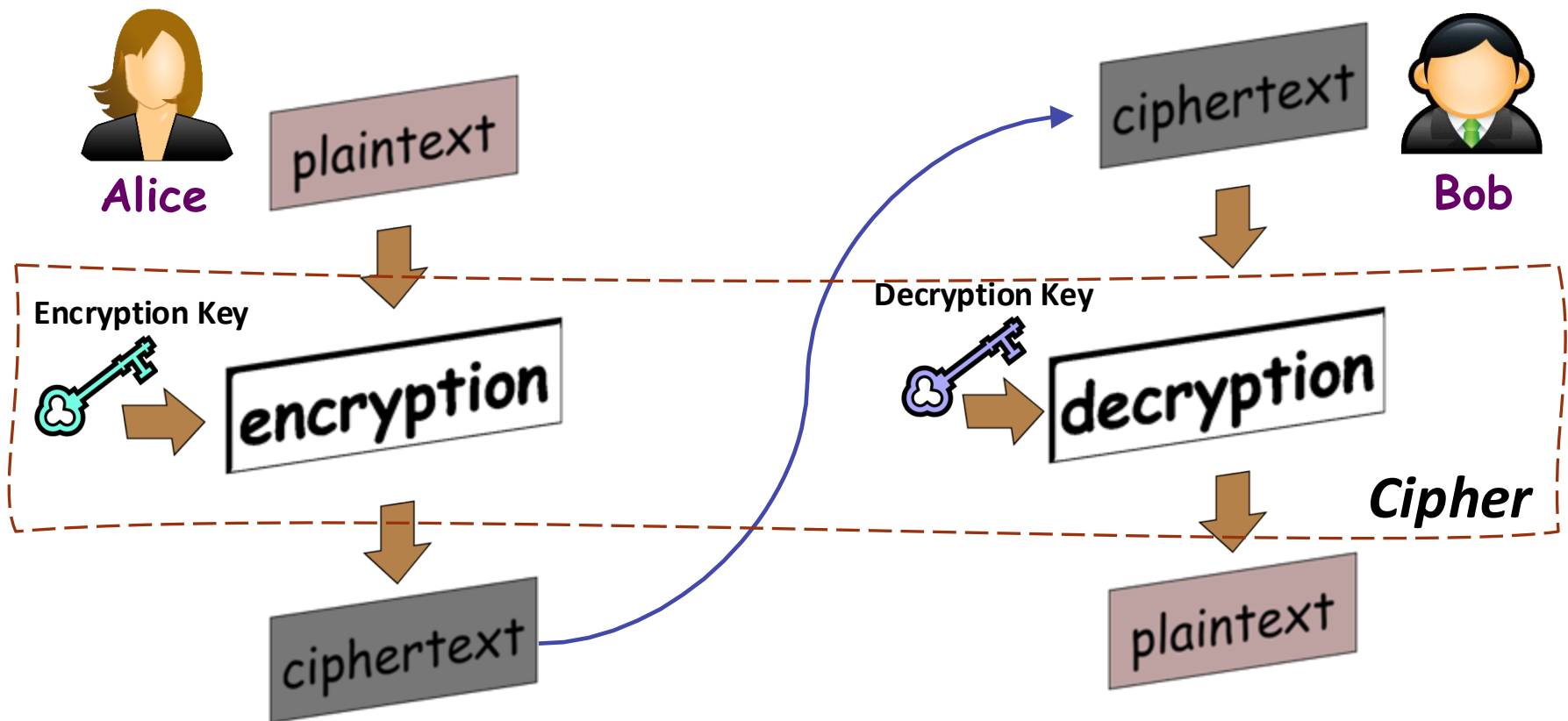
- Coding Practice
- How to Secure Applications

Cryptography

**Secure
Development**

I. Cryptography

How does Cryptography work?



Encryption Schemes

► Two main schemes:

1. Single-key/Symmetric cryptosystems

- Stream Cipher: Transposition and Substitution
- Block cipher: Feistel algorithms, DES, and AES

2. Public-key/Asymmetric cryptosystems

- Diffie Hellman
- RSA
- Elliptic Curve Cryptosystem

Symmetric Ciphers: Stream Ciphers

- Transposition ciphers:
 - Split characters into blocks of fixed length d .
 - Rearrange the characters inside a block according to a key dependent permutation.
 - Decryption uses the inverse permutation to recover the plaintext.

- Substitution ciphers:
 - Provide a matching between letters in the plaintext alphabet to other letters in the same or a different alphabet for the ciphertext.
 - Shift Substitution
 - Multiplicative Substitution

Symmetric Ciphers: Block Ciphers

Encrypt a block of plaintext (L_0, R_0) :

- ▶ For each round $i = 0, 1, 2, \dots, n$:
 - Input: L_i , R_i , and K_i (round key)
 - Output: L_i and R_i
 - A substitution is performed on the left half L_i

$$L_i \oplus F(R_i, K_i)$$

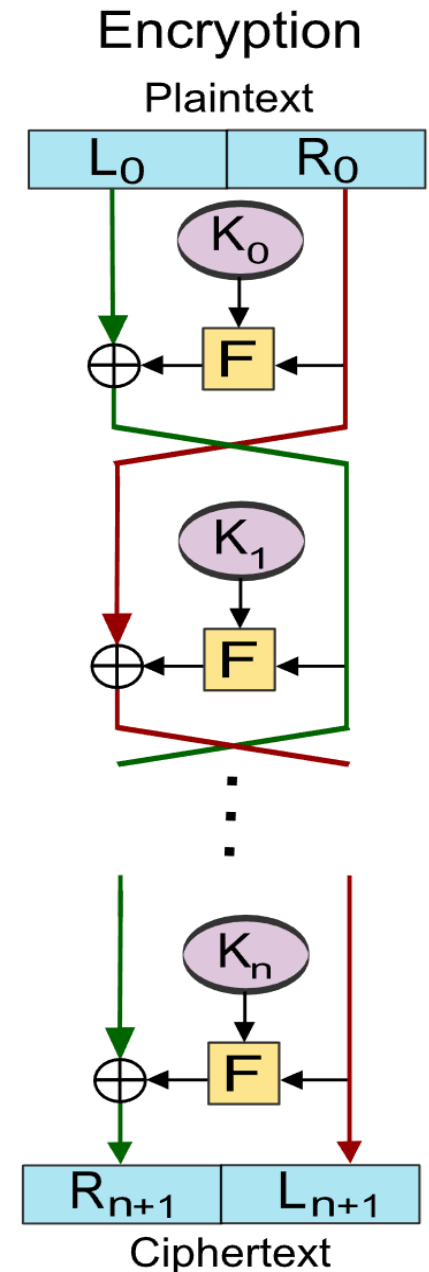
- A permutation is performed by swapping the two halves.

$$L_{i+1} = R_i$$

$$R_{i+1} = L_i \oplus F(R_i, K_i)$$

Then the ciphertext is (R_{n+1}, L_{n+1})

- ▶ F is any function, there are several different Feistel algorithm, each with a different function F .
- ▶ K_i is a round key (i.e., sub-key), $\oplus$ is exclusive OR.



https://en.wikipedia.org/wiki/Feistel_cipher

Public-Key Cryptography

Symmetric Key Crypto

Key Distribution challenge:
Requires sender, receiver know
shared secret key:

Q: How to agree on key in first
place (particularly if never
“met”)?

Public Key Cryptography

- Radically different approach
[Diffie-Hellman76, RSA78]
- Sender, receiver do *not*
share secret key
- *Public* encryption key
known to *all*
- *Private* decryption key
known only to receiver



Key Exchange challenge

- One of the main problems of symmetric key encryption is it requires a secure & reliable channel for the shared key exchange.
- The Diffie-Hellman Key Exchange protocol offers a way in which a public channel can be used to create a confidential shared key
- **Diffie-Hellman Key Exchange:** a protocol that enables two users to establish a secret key using a public-key scheme based on discrete logarithms.
 - The protocol is secure only if the authenticity of the two participants can be established.
- In practice the shared encryption key relies on such complex concepts as *Modular Exponentiation*, *Primitive Roots* and *Discrete Logarithm Problems*

Diffie-Hellman Summary

- Multiplicative group of “Integers modulo p ”

$$s = g^n \bmod p$$

- We have:

Easy: given g , n , & p , solve for s

Hard: given s , g , & p , solve for n

- And the property of:

$$g^{a*b} \bmod p = g^{b*a} \bmod p$$

RSA Summary

- ▶ RSA algorithm

- ▶ How to generate the key pair

Public key: $pk = (e, n)$; Private key: $sk = (d, n)$

- ▶ How to encrypt using the public key

Compute ciphertext $C = M^e \bmod n$

- ▶ How to decrypt using the private key

Compute $M' = C^d \bmod n$

$$= (M^e \bmod n)^d \bmod n$$

$$= M^{ed} \bmod n = M$$

Elliptic Curves Crypto

➤ Elliptic Curves Cryptography

- Basic Elliptic Curves

$$y^2 = x^3 + Ax + B$$
$$(4A^3 + 27B^2 \neq 0) \text{ (Weierstrass Form)}$$

- Elliptic Curve for Cryptography = Elliptic Curve over a Finite Field L : $E(L)$

$$E(L) = \{\infty\} \cup \{(x, y) \in L \times L \mid y^2 + \dots = x^3 + \dots\}$$

- Combine with “*Integer mod p*”, P is a prime number to form a Finite Field

Elliptic Curves Crypto

➤ Elliptic Curves Cryptography

- Addition and Multiplication on Elliptic Curve
 - How we define $P + Q$ over $E(L)$
 - How we define $mP = P + P + \dots + P$ over $E(L)$
- Elliptic Curves Discrete Logarithm Problem (ECDLP)
- Elliptic Curves Diffie Hellman (ECDH)

Elliptic Curves Crypto

➤ Elliptic Curves Discrete Logarithm Problem (ECDLP)

- There is efficient way to calculate the multiplication on EC:

$$Q = mP, \text{ given } m \text{ and } P$$

- The ECDLP is to find the value of m , given Q and P that

$$Q = mP$$

- Basically, we do not know of a fast way to solve ECDLP.
- This will be good for the trapdoor function (with modular p)
 - m will be the secret information, Q and P and the modulus prime p can be publicly available

Elliptic Curves Crypto

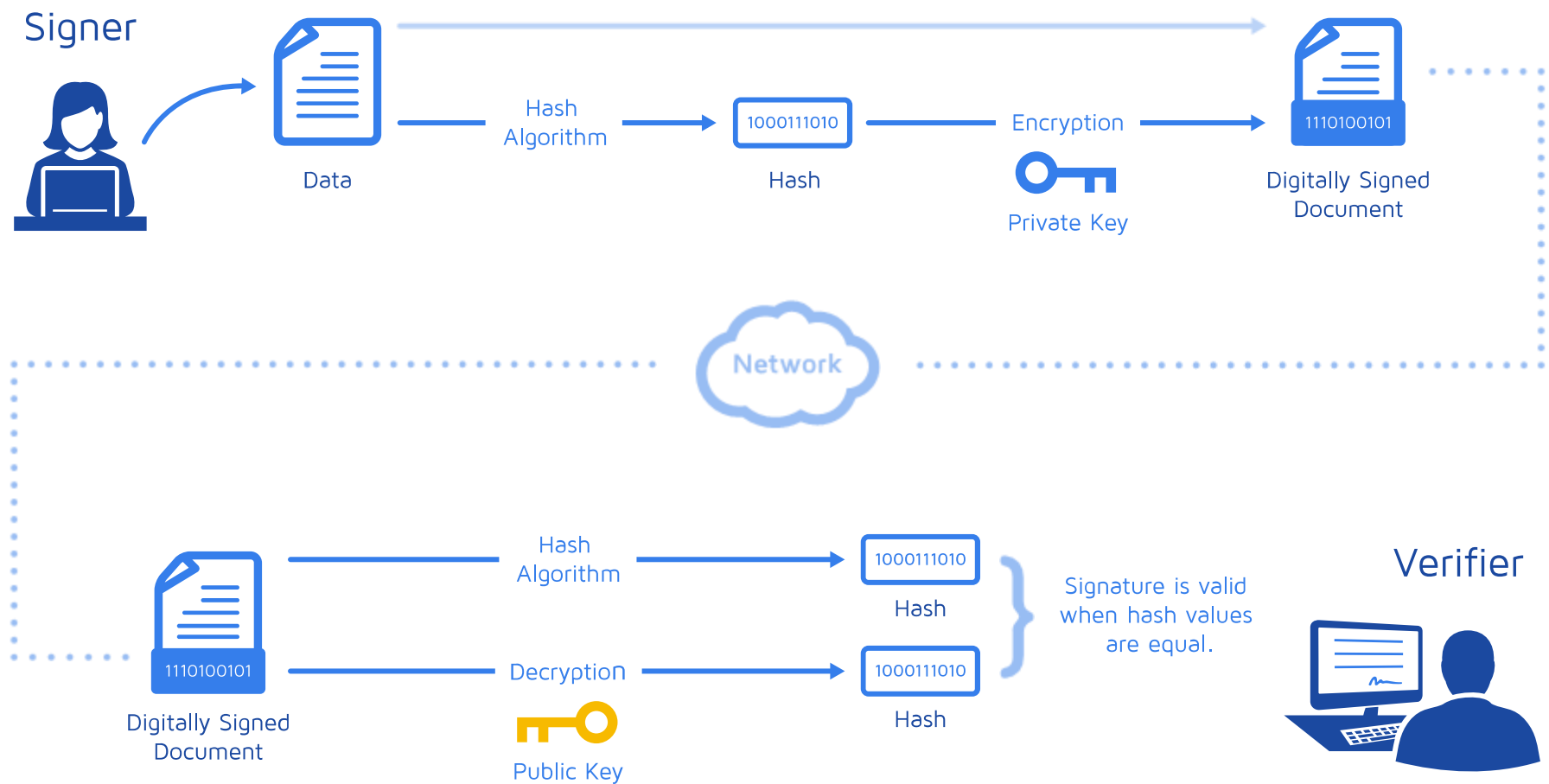
➤ Elliptic Curves Diffie Hellman Key Exchange

- Alice and Bob desire to establish a shared secret key k .
- Alice and Bob agree on an EC, a large prime p (called the multiplicative order), and a point B on the curve (called the Generator or the Base-point)
 - Called Domain Parameters
- Eve knows the curve, the point, and the prime number

Elliptic Curves Crypto

- Alice secretly picks a large number p_A . Bob secretly picks a large number p_B . Alice computes $Q_A = p_A B$. Bob computes $Q_B = p_B B$. They exchange the points Q_A and Q_B .
 - p : private key
 - $Q = pB$: public key
- Alice computes $p_A Q_B = p_A p_B B$. Bob computes $p_B Q_A = p_B p_A B$. Both use the x value of $p_A p_B B$ for the key k .
- The shared secret key k is then used for symmetric encryption such as DES and AES

How does Digital Signatures work ?



Digital Signature Scheme

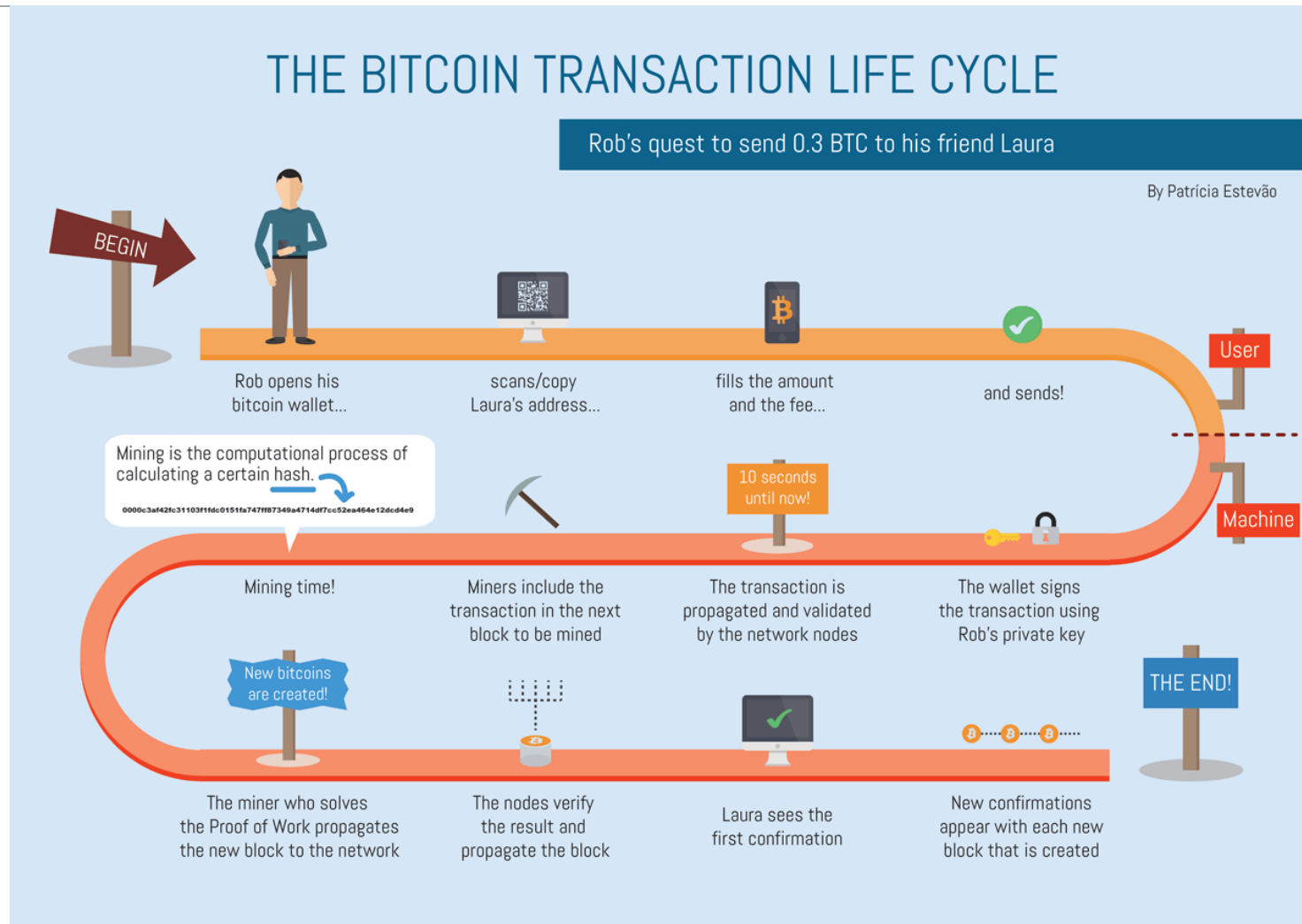
- Digital Signature: a data string which associates a message with some originating entity. One party generates a signature, many parties can verify.
- Digital Signature Scheme:
 - a signing algorithm: takes a message and a (private) signing key, outputs a signature
 - a verification algorithm: takes a (public) key verification key, a message, and a signature
- Provides:
 - Authentication, Data integrity, Non-Repudiation

II. Secure Development

BitCoin & Cryptocurrencies

1. Bitcoin & Cryptocurrencies
 - Decentralised Payment System: A Big Picture
 - Cryptography in Bitcoin
 - Bitcoin Protocol: an Overview
2. Bitcoin Protocol and Blockchain Technology
 - Bitcoin Addresses
 - Bitcoin Distributed Ledger: a Blockchain
 - Transactions in Bitcoin and Blockchain
 - Consensus Mechanism: Proof-of-Work
3. Security and Privacy in Bitcoin and Blockchain

Bitcoin Transaction Lifecycle



<https://www.bambora.com/en/ca/blog/bitcoin-explained/>

Outline

1. SQL Injection

2. Cross-Site Scripting XSS

3. Programming Languages Problem

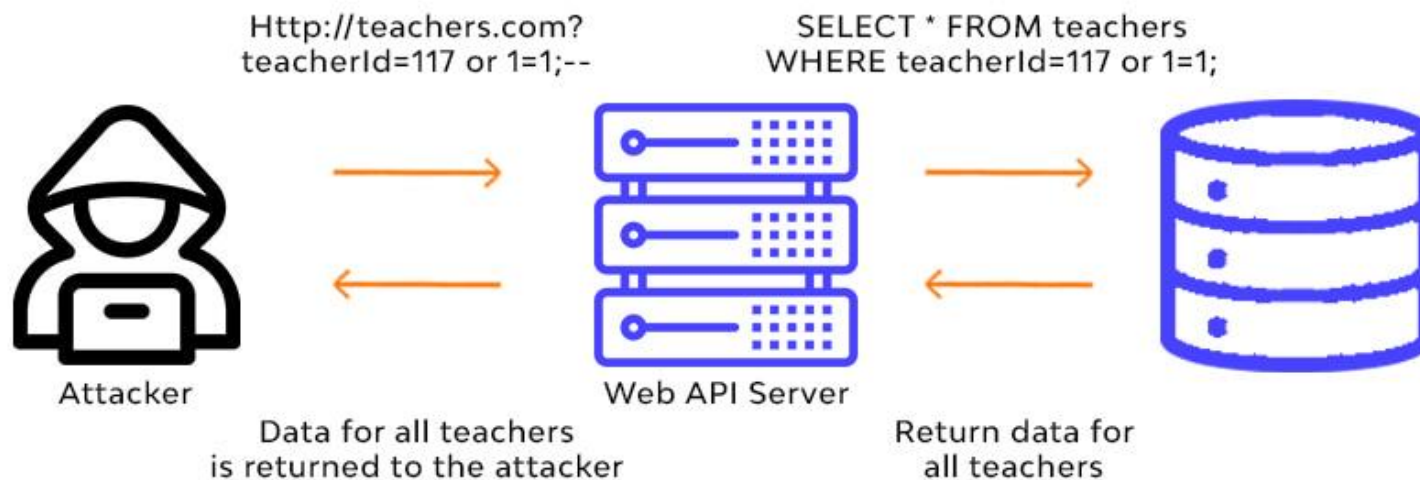
- C/C++ Buffer Overflow
- Poisoned Null Byte / Null Byte Injection

4. Java Security

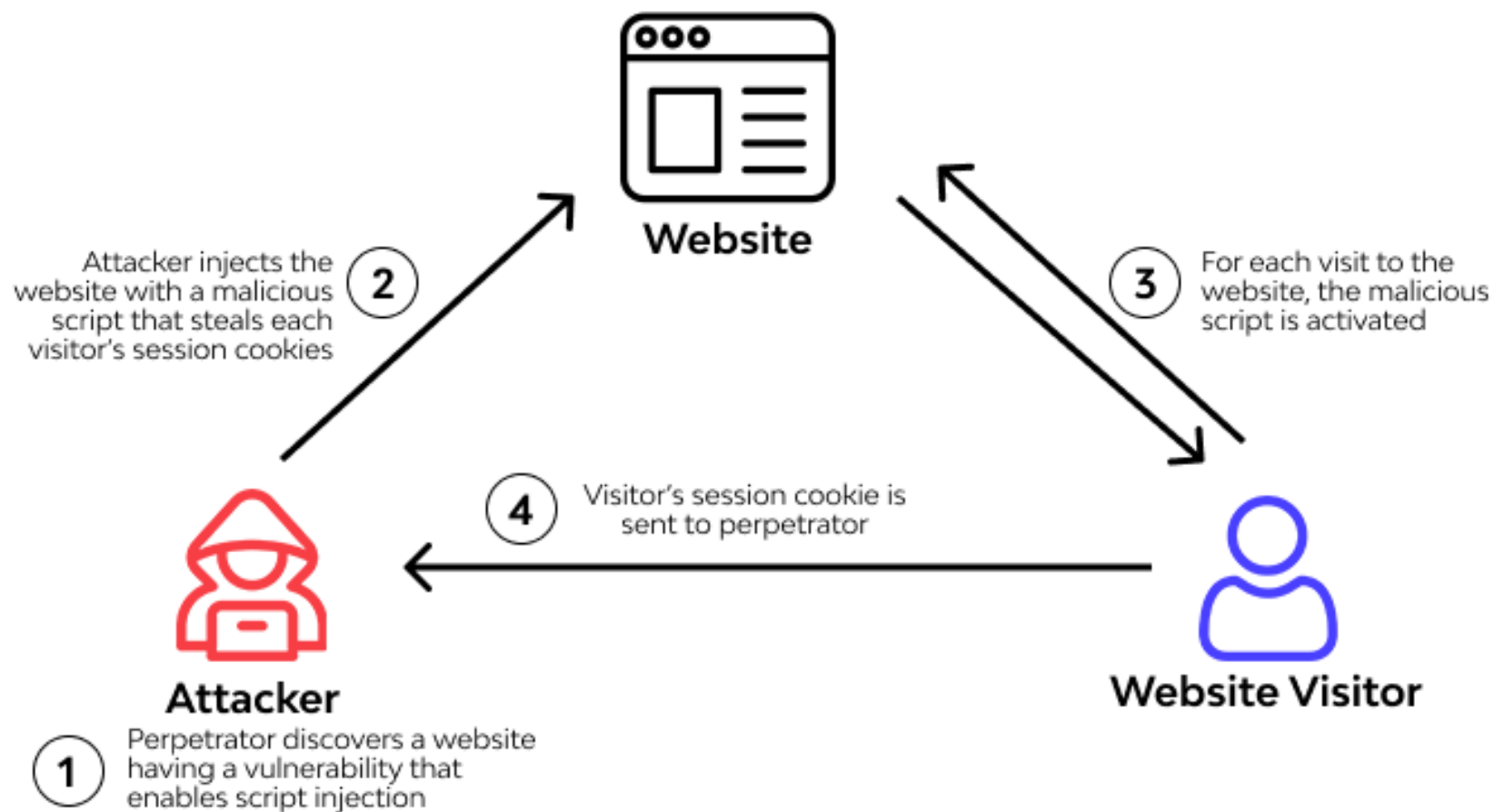
5. Java Security Demo

SQL Injection

SQL Injection



Stored cross-site scripting



JAVA Standard Crypto Libraries

- It is a bad idea to implement our own crypto algorithms.
- Building a secure system may need to use standard crypto algorithms.
- We do this by using a library of supplied crypto implementations.
- Java has its own crypto packages:
 - `java.security.*`
 - `javax.crypto.*`
 - `javax.crypto.spec.*`

AES Key Generation

- From `javax.crypto` library.
- `KeyGenerator` is used to generate a `SecretKey`.
 - There is only one key for AES encryption and decryption and so it must be kept secret.
- It is generated by the `getInstance` factory method, specifying AES.
- The key length is provided in the `init` method.
- The key is created by the `generateKey` method.
- `Myprint_bytes` method will print out a byte array in hexadecimal.
- There is an exception to be caught (see full listing).

AES Encryption/Decryption

- `javax.crypto` has an abstract class called `Cipher`.
- Use the `getInstance` factory method specifying an AES cipher.
- Initialise it by calling the `init` method with the mode and key as parameters.
 - There are 2 possible modes.
 - `Cipher.ENCRYPT_MODE`
 - `Cipher.DECRYPT_MODE`
- Finally, the encryption is done by the `doFinal` method.
- Long files need several calls to `update` before `doFinal`.
- There are some exceptions (not included for brevity).

Exam Format and Mock Questions

Exam Format

- The exam is open-book
- You attend the exam in a school exam room.
- The exam lasts **2 hours** consisting of **4 BIG questions**

Exam Format

- The 4 questions are in the following structure:
 - **Question 1: Classical Cryptography**
 - Just some simple calculations but require you to understand the algorithm. For instance, the Substitution algorithms, etc.
 - **Question 2: Symmetric-key Cryptography**
 - Simple calculations but require you to understand the algorithm. For instance, Feistel, DES, and AES algorithms, etc.
 - **Question 3: Asymmetric Cryptography (Elliptic Curves)**
 - Just simple calculations but require you to understand the algorithm. For instance, the Diffie Hellman, RSA, etc.
 - How to define an ECC, how to calculate the Addition of two point or Doubling a point, ect.

Exam Format

➤ Question 4: Secure Development

- Bitcoin/Blockchain understanding
- Ask to implement secure mechanisms (in Java Programming Language)
- You do not need to write the whole code, just write the program structure and class declaration with libraries.

Some sample questions

One-way Operation

- We need to find an operation similar to “Mixing colours”: Easy to DO and hard to UNDO. Let’s consider Multiplicative group of Integers mod p

$$s = g^n \text{ mod } p$$

- We have:

Easy: given g , n , & p , solve for s

Hard: given s , g , & p , solve for n

- And the property of:

$$g^{a*b} \text{ mod } p = g^{b*a} \text{ mod } p$$

Step 1-Publicly shared information

- Alice & Bob publicly agree to a large prime number called the modulus, or p .
- Alice & Bob publicly agree to a number called the generator, or g , which has a primitive root relationship with p .
- In our example we'll assume $p = 17, g = 3$
- Trudy is aware of the values of p or g .

RSA Algorithm

- Invented in 1978 by Ron Rivest, Adi Shamir and Leonard Adleman
- Published as R L Rivest, A Shamir, L Adleman, "On Digital Signatures and Public Key Cryptosystems", Communications of the ACM, vol 21 no 2, pp120-126, Feb 1978.
- Security relies on the difficulty of factoring large composite numbers
- Essentially the same algorithm was discovered in 1973 by Clifford Cocks, who works for the British intelligence

RSA Example

Alice wants to send Bob a message M using RSA

1. Bob generates a key-pair (pk, sk) ; pk is publicly available.
2. Alice encrypts the message M with Bob's public-key pk
3. Bob decrypts the ciphertext C with his private-key sk

RSA Example: Key generation

Key generation: how to generate (pk, sk) for Bob

1. Select 2 large prime numbers of about the same size, p and q
2. Compute $n = pq$, and $\Phi(n) = (q-1)(p-1)$
3. Select e , $1 < e < \Phi(n)$, s.t. $\gcd(e, \Phi(n)) = 1$
4. Compute d , $1 < d < \Phi(n)$ s.t. $ed \equiv 1 \pmod{\Phi(n)}$

Then the key pair is formulated as follows:

Public key: $pk = (e, n)$; Private key: $sk = (d, n)$

RSA Example:

Encryption and Decryption

Encryption:

Given a message M , $0 < M < n$; $M \in \mathbb{Z}_n - \{0\}$

Use the public key $pk = (e, n)$

Compute ciphertext $C = M^e \bmod n$; $C \in \mathbb{Z}_n - \{0\}$

Decryption:

Given a ciphertext C

Use the private key $sk = (d, n)$

Compute $C^d \bmod n = (M^e \bmod n)^d \bmod n = M^{ed} \bmod n = M$

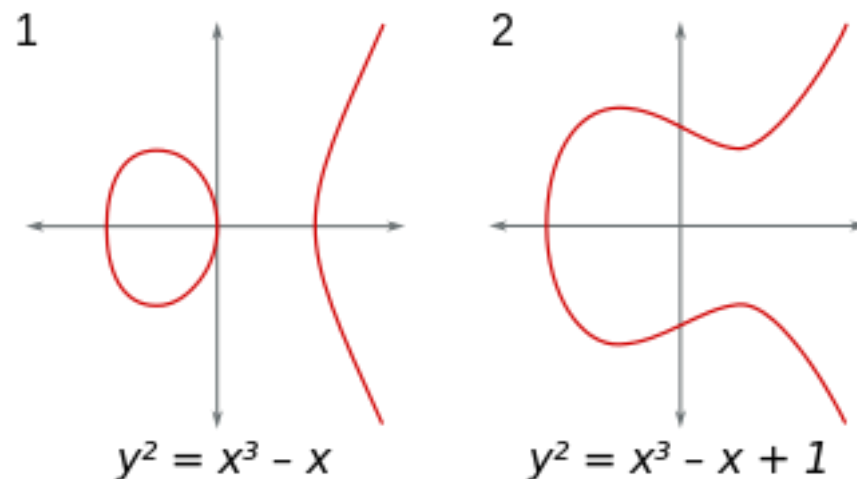
Elliptic Curves

➤ Basic Elliptic Curves:

$$y^2 = x^3 + Ax + B$$

$(4A^3 + 27B^2 \neq 0)$ (Weierstrass Form)

➤ The values A, B, x, y come from some set, usually a field



Definition of EC

- An **elliptic curve** over a field L is a non-singular cubic curve in two variables, $f(x,y) = 0$ with a rational point (which may be a point at infinity).
- The field L is usually taken to be the complex numbers, reals, rationals, algebraic extensions of rationals, p-adic numbers, or a **finite field**.
- **Elliptic curves groups for cryptography** are examined with the underlying fields of F_p (where $p > 3$ is a prime) and F_{2^m} (*a binary representation with 2^m elements*).

EC on a Finite Set of Integers

➤ Consider $y^2 = x^3 + 2x + 3 \pmod{5}$

$$x = 0 \Rightarrow y^2 = 3 \Rightarrow \text{no solution} \pmod{5}$$

$$x = 1 \Rightarrow y^2 = 6 = 1 \Rightarrow y = 1, 4 \pmod{5}$$

$$x = 2 \Rightarrow y^2 = 15 = 0 \Rightarrow y = 0 \pmod{5}$$

$$x = 3 \Rightarrow y^2 = 36 = 1 \Rightarrow y = 1, 4 \pmod{5}$$

$$x = 4 \Rightarrow y^2 = 75 = 0 \Rightarrow y = 0 \pmod{5}$$

➤ Then points on the elliptic curve are: $(1, 1)$ $(1, 4)$ $(2, 0)$ $(3, 1)$ $(3, 4)$ $(4, 0)$ and the point at infinity: ∞

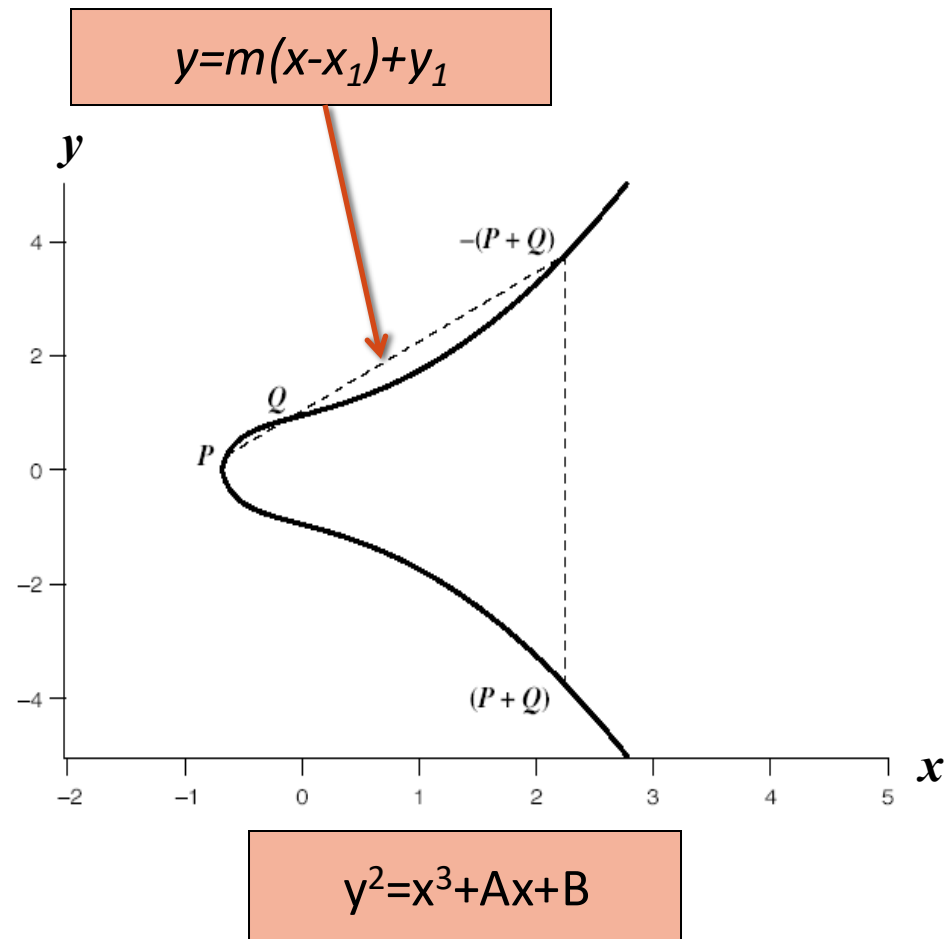
➤ Using finite fields, we can form an Elliptic Curve Group for public-key cryptography where we have a trapdoor function as the DLP problem.

Addition of two Points on EC

Consider two points P and Q

$$P = (x_1, y_1), Q = (x_2, y_2)$$

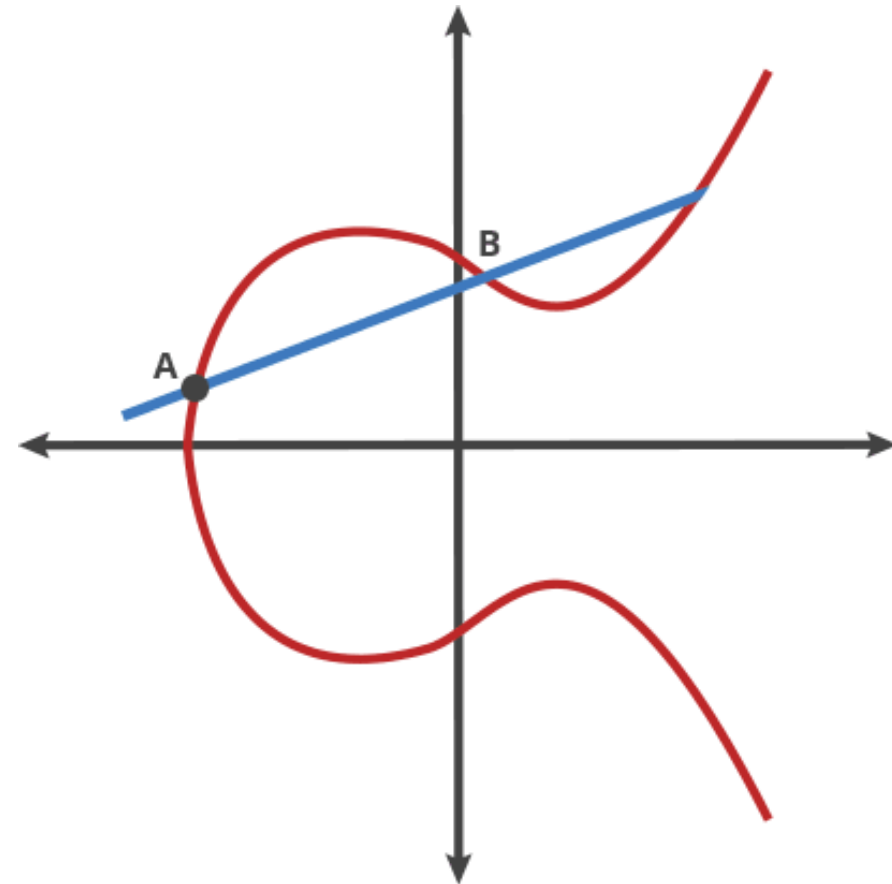
$$R = (P + Q) = (x_3, y_3)$$



Addition of two Points on EC

➤ Using our definition of addition:

1. $P + Q$ is well defined
2. $P + Q = Q + P$
3. $P + (Q + R) = (P + Q) + R$
4. $P + O = P$
5. $-P = P \# (O \# O)$



<https://blog.cloudflare.com/a-relatively-easy-to-understand-primer-on-elliptic-curve-cryptography>

Doubling a Point on EC

- In case $P = Q$

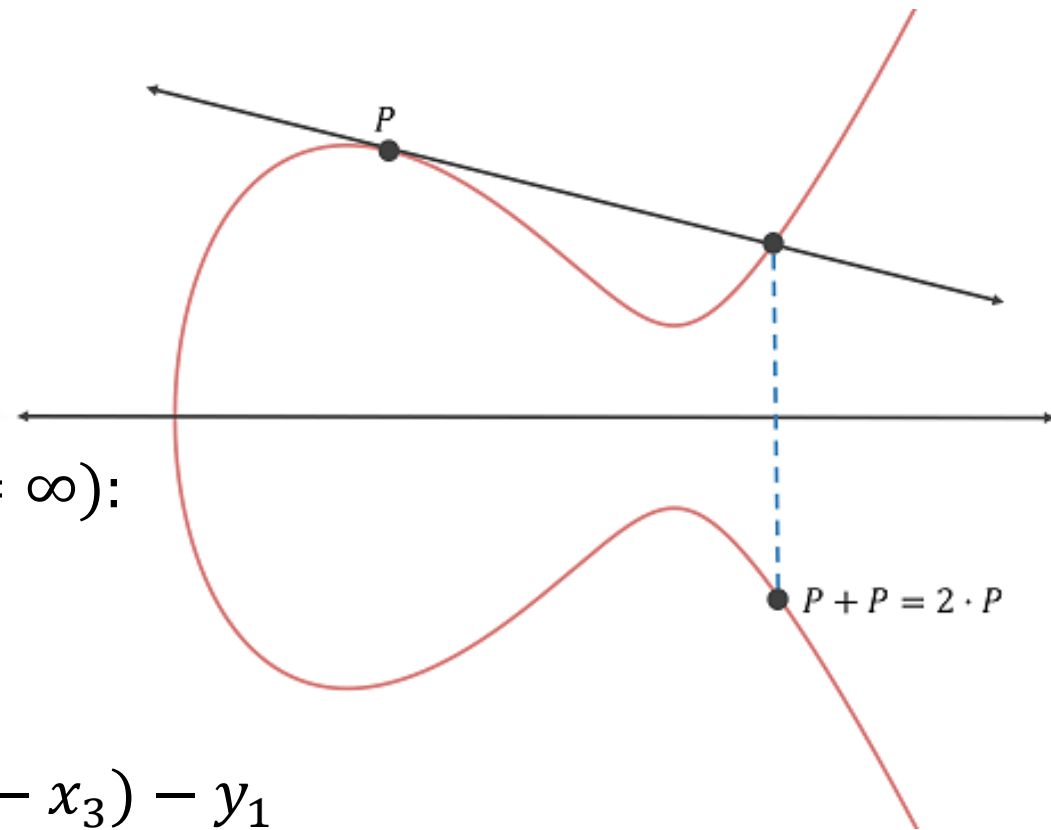
$$2y \frac{dy}{dx} = 3x^2 + A$$

$$\Rightarrow m = \frac{dy}{dx} = \frac{3x_1^2 + A}{2y_1}$$

If, $y_1 \neq 0$ (since then $P_1 + P_2 = \infty$):

$$0 = x^3 - m^2 x^2 + \dots$$

$$\Rightarrow x_3 = m^2 - 2x_1, y_3 = m(x_1 - x_3) - y_1$$



Multiplication of a Point

We define mP :

$$2P = P + P$$

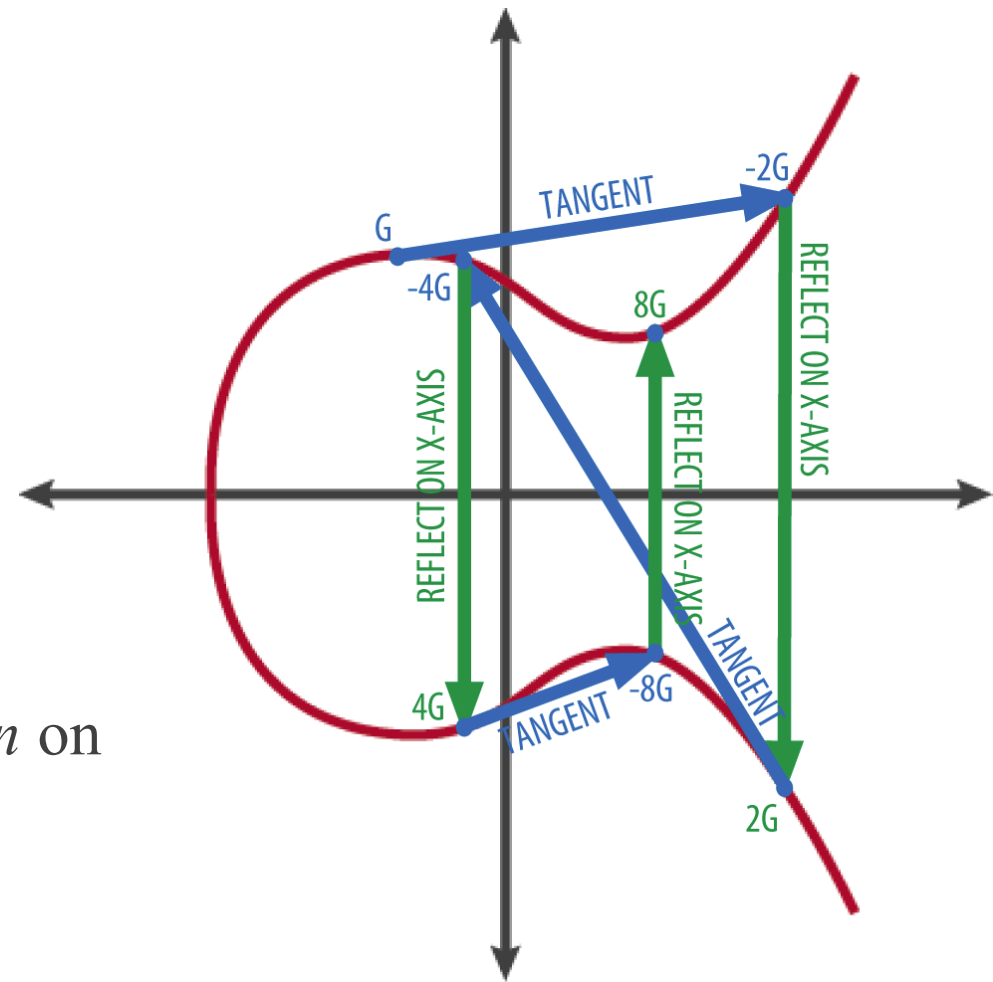
$$3P = P + P + P$$

$$mP = P + P + \dots + P$$

m : order of P

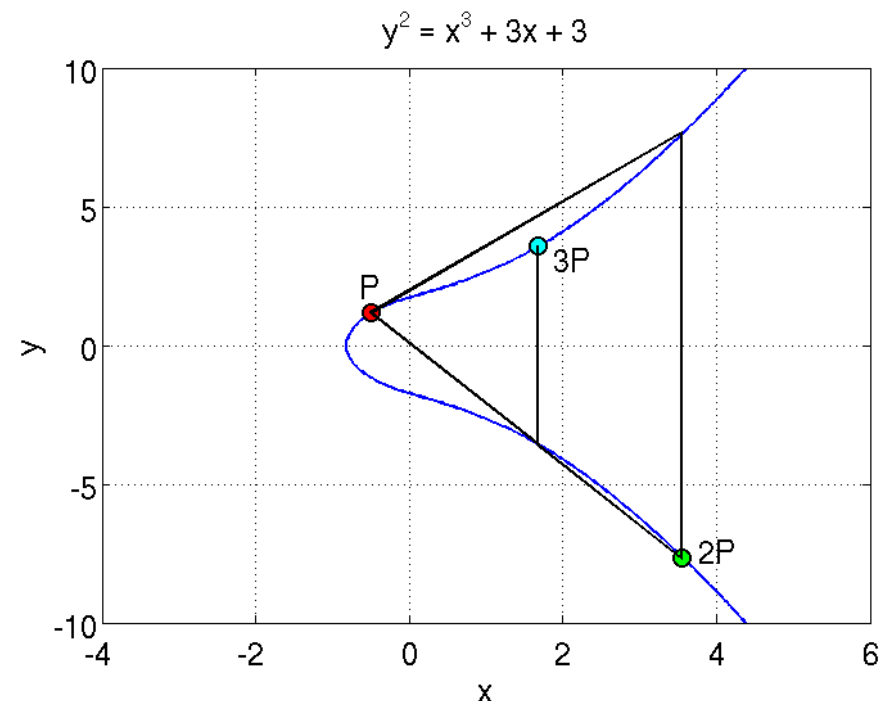
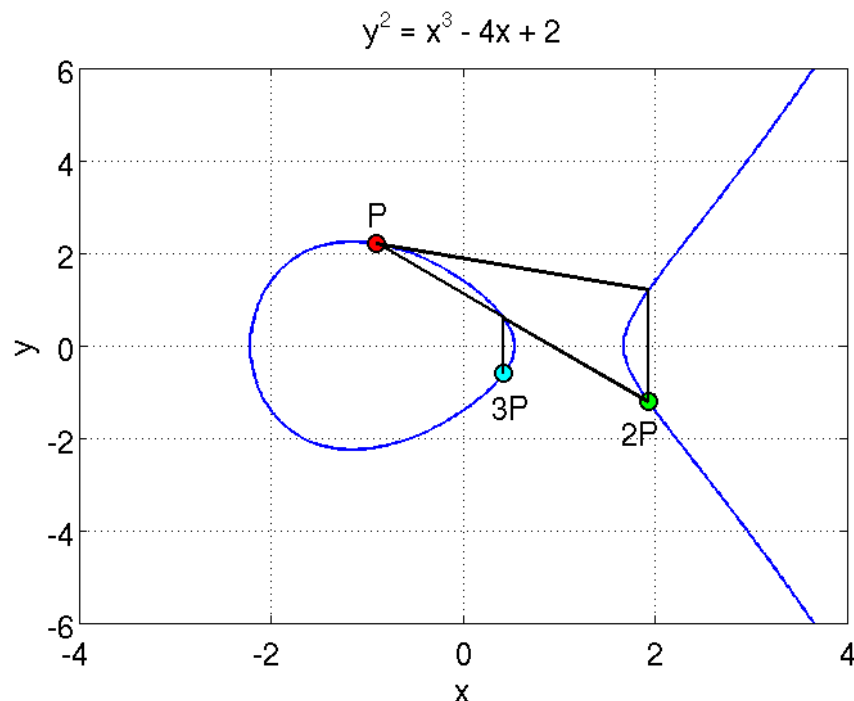
No matter how big m is, there is an efficient way to calculate mP .

It is also called *Scalar Multiplication* on EC



Multiplication of a Point

- A tangent at P is extended to cut the curve at a point; its reflection is $2P$. Adding P and $2P$ gives $3P$
- Similarly, such operations can be performed as many times as desired to obtain $Q = mP$



EC Discrete Log Problem

- There is efficient way to calculate the multiplication on EC: $Q = mP$, given m and P
- The ECDLP is to find the value of m , given Q and P that $Q = mP$
 - Basically, we do not know of a fast way to solve ECDLP.
- This will be good for the trapdoor function (with modular p)
 - m will be the secret information, Q and P and the modulus prime p can be publicly available

Main points to take home

▶ Digital Signature based on RSA:

A triple (Gen, Sign, Ver)

- ▶ Gen(len) : generate a key-pair (pk, sk)
- ▶ Sign(m; sk) : sign a message m with private key sk, output signature s
- ▶ Ver(m, s; pk) : verify signature s with public key pk

Main points to take home

▶ Digital Signature based on RSA:

A triple (Gen, Sign, Ver)

- ▶ Gen(len) : generate a key-pair (pk, sk)
- ▶ Sign(m; sk) : sign a message m with private key sk, output signature s
- ▶ Ver(m, s; pk) : verify signature s with public key pk

$$(a^2)/c*(b^3 + 4)$$

$$a^{(-1)}$$

$$(a/b) \bmod p = a*(b^{(-1)}) \bmod p$$

$$\text{Invert}(b)$$